

Angular 22 --- Parte 2

Formularios, Guards, Signals, Testing y Despliegue

Continuacion del bloque anterior --- del setup y consumo REST al proyecto listo para produccion

Requisito previo: haber completado el deck 1 (entorno instalado, proyecto frontend creado, consumo REST basico).

Que se anade: formularios reactivos con Signal Forms, protecciones de rutas, signals derivadas,
pruebas con Vitest y despliegue en Docker con nginx.

Stack: Angular 22.0.5 | @angular/forms/signals | Vitest | Docker 24+ | nginx 1.27+

Recapitulacion del bloque anterior

Antes de avanzar, se resume que se dejo funcionando en el deck 1 para asegurar que el aprendiz parta del mismo punto que el docente.

- Node.js 24 LTS instalado y verificado con `node --version` (comando que imprime la version del interprete de JavaScript del servidor).
- Angular CLI 22 instalado globalmente con `npm install -g @angular/cli@22` (herramienta de linea de comandos que crea y mantiene proyectos Angular).
- IntelliJ IDEA Ultimate configurado con plugins Angular y TypeScript activos.
- Proyecto frontend/ generado con `ng new frontend` (comando que crea un proyecto Angular completo con enrutador y estilos CSS).
- Componentes standalone y servicios inyectables (`@Injectable`) generados con `ng generate`.
- HttpClient registrado con `provideHttpClient()` y consumo de `/api/libros` implementado (metodos GET, POST, PUT, DELETE del CRUD).

Comprobacion rapida

Ejecutar `'ng serve --open'` en la carpeta frontend/ debe abrir localhost:4200 y mostrar el catalogo de libros consumido desde el backend en localhost:8080. Si no es asi, revisar el deck 1 antes de continuar.

Objetivos de la continuacion

- Construir formularios con Signal Forms, la API estable en Angular 22 basada en signals (contenedores reactivos de estado).
- Aprender los signals derivados con `computed()` (funcion que calcula un valor a partir de otros signals) y los efectos con `effect()` (funcion que se dispara cuando algun signal leído cambia).
- Proteger rutas con guards funcionales, funciones que Angular ejecuta antes de activar una ruta y que deciden si el usuario puede acceder.
- Cargar componentes bajo demanda con `loadComponent` (mecanismo de lazy loading, o carga diferida, que reduce el bundle inicial).
- Escribir pruebas unitarias con Vitest, el corredor de pruebas por defecto de Angular desde la version 21, y con TestBed, la utilidad oficial de Angular para montar componentes en un entorno de prueba.
- Empaquetar el frontend en un Dockerfile multi-stage y desplegarlo detras de nginx con `proxy_pass` hacia la API de BiblioUTEQ.

Formularios: dos APIs coexisten

Termino: Formulario

Vista con campos (input, select, textarea) que capturan datos del usuario y los envian al servidor. Un formulario tiene estado (valores actuales, si se toco, si es valido, si tiene errores) y comportamiento (validacion, envio).

En Angular 22 conviven dos APIs oficiales para escribir formularios. Cual se usa depende del proyecto y del gusto del equipo; ambas tienen soporte a largo plazo, segun la nota oficial del anuncio de Angular 22.

Reactive Forms (clasico)

Modulo: `@angular/forms`. Se construye con clases `FormGroup` y `FormControl`. Cada control es un objeto que expone `valueChanges` (un `Observable`) y `statusChanges`. Ha sido el estandar desde Angular 2 (2016). Requiere gestionar suscripciones y `ControlValueAccessor` para inputs personalizados.

Signal Forms (estable en v22)

Modulo: `@angular/forms/signals`. Se construye con la funcion `form()` a partir de una signal que contiene los datos. Cada campo es una signal tipada. Todo se lee y se muta a traves de signals; no hay `Observables` ni `subscribe`. Se recomienda para formularios nuevos.

Signal Forms: piezas clave

Termino: Signal

Contenedor reactivo de un valor. Se lee llamandolo como funcion (`mi.signal()`) y se muta con `set()` o `update()`. Cuando cambia, Angular re-renderiza automaticamente las vistas y calcula lo derivado que dependa de el.

`signal<T>(inicial)`

Crea la signal que guarda el modelo del formulario. Su tipo T describe la forma del objeto (ej. interface Libro).

`form(mi_signal)`

Convierte la signal en un FieldTree: arbol donde cada campo del modelo es una signal con estado (valor, valido, tocado, errores).

`[formField]="f.campo"`

Directiva que enlaza un `<input>` con la rama del arbol; sincroniza valor y estado en ambas direcciones.

`submit(form, fn)`

Ejecuta la accion de envio; marca el formulario como enviado y solo procesa si `validity()` vale 'valid'.

Signal Forms: no requiere instalacion

El modulo `@angular/forms/signals` ya viene incluido en el paquete `@angular/forms` que la CLI instala por defecto. Basta importarlo en el componente que lo use.

```
imports base
```

```
// biblioteg-front/src/app/libros/libro-form.component.ts

// Component: decorador que declara la clase como componente Angular.
// signal: fabrica la senal que guarda el modelo del formulario.
// inject: helper que obtiene una instancia registrada del inyector.
import { Component, signal, inject } from '@angular/core';
// form: crea el FieldTree a partir de una signal.
// FormField: directiva de plantilla que enlaza <input> con la rama.
// submit: helper que dispara el envio y valida antes de ejecutar el callback.
import { form, FormField, submit } from '@angular/forms/signals';
import { LibroService } from '../libro.service';
import { Libro } from '../libro.model';
```

Signal Forms: formulario minimo

libro-form.component.ts

```
// Decorador @Component: metadatos de configuracion del componente.
@Component({
  selector: 'app-libro-form',          // etiqueta HTML del componente
  standalone: true,                   // no requiere NgModule (default en v22)
  imports: [FormField],               // directiva formField usada en la plantilla
  templateUrl: './libro-form.html'   // ruta al archivo de plantilla externa
})

export class LibroFormComponent {
  // signal<Libro>() : contenedor reactivo con el modelo inicial vacio.
  // Su valor se lee con model() y se muta con model.set() o model.update().
  protected readonly model = signal<Libro>({
    id: 0, isbn: '', titulo: '', autor: '',
    editorial: '', anio: 2026, categoriaId: 1, disponibles: 1
  });

  // form() : convierte la signal en un FieldTree.
  // libroForm.titulo, libroForm.isbn... son signals de campo con estado.
  protected readonly libroForm = form(this.model);
}
```

Signal Forms: plantilla del formulario

libro-form.html

```
<!-- libro-form.html: enlaza cada input a una rama del FieldTree -->
<form (submit)="guardar($event)">
  <!-- [formField] es una directiva estructural que sincroniza -->
  <!-- el valor del input con la signal del campo del arbol. -->
  <label>Titulo
    <input [formField]="libroForm.titulo" />
  </label>

  <!-- errors() es una signal que devuelve la lista de errores -->
  <!-- de un campo; @if la lee y muestra un mensaje si hay uno. -->
  @if (libroForm.titulo.errors().length > 0) {
    <p class="error">Titulo obligatorio.</p>
  }

  <label>ISBN <input [formField]="libroForm.isbn" /></label>
  <label>Autor <input [formField]="libroForm.autor" /></label>
  <button type="submit">Guardar</button>
</form>
```


Validacion declarativa con un schema

schema declarativo

```
// schema() : recibe una funcion (path) => que declara las reglas
// para cada rama. path apunta al arbol de campos del modelo.
import { form, schema, required, minLength, maxLength, min, max }
  from '@angular/forms/signals';

// El schema es un valor declarativo: no ejecuta validaciones,
// las anota. La validacion se corre sola cuando cambia el valor.
const libroSchema = schema<Libro>((path) => {

  // required : el campo no puede ser cadena vacia ni undefined.
  required(path.titulo, { message: 'El titulo es obligatorio' });
  required(path.isbn, { message: 'El ISBN es obligatorio' });
  required(path.autor, { message: 'El autor es obligatorio' });

  // minLength / maxLength : longitud del texto en caracteres.
  minLength(path.titulo, 3);
  maxLength(path.titulo, 120);

  // min / max : rango numerico. Se aplica al campo anio.
  min(path.anio, 1500);
  max(path.anio, 2100);
});

// Al declarar el formulario, se pasa el schema como segunda opcion:
protected readonly libroForm = form(this.model, libroSchema);
```

Envío del formulario (submit)

guardar()

```
// En la clase del componente:
export class LibroFormComponent {

  // inject() : obtiene la instancia unica del servicio del inyector.
  // Es equivalente moderno de tener el servicio como parametro del constructor.
  private readonly servicio = inject(LibroService);
  protected readonly model = signal<Libro>({...});
  protected readonly libroForm = form(this.model, libroSchema);

  // guardar() : callback del evento (submit) del <form>.
  // event.preventDefault() : evita la recarga por defecto del navegador.
  // submit() : helper de Signal Forms que marca el form como enviado,
  //           valida, y solo ejecuta la funcion callback si es valido.
  guardar(event: Event): void {
    event.preventDefault();
    submit(this.libroForm, async (form) => {
      // form.value() : signal que expone el modelo completo tipado.
      // subscribe() : consume el Observable devuelto por HttpClient.
      this.servicio.crear(form.value()).subscribe({
        next: (libroCreado) => console.log('OK', libroCreado.id),
        error: (err) => console.error('Error al crear', err)
      });
    });
  }
}
```

Reactive Forms como alternativa

Reactive Forms sigue soportado y aparece en la mayoría de proyectos existentes. Se resume aquí para que el aprendiz reconozca su sintaxis, no para recomendarla en proyectos nuevos.

Reactive Forms

```
// FormBuilder : servicio inyectable que fabrica controles/grupos.
// ReactiveFormsModule : modulo que expone las directivas FormGroup/FormControlName.
// Validators : coleccion de funciones estaticas (required, email, minLength).
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';

@Component({ imports:[ReactiveFormsModule], /* ... */ })

export class LibroFormReactivoComponent {
  private fb = inject(FormBuilder);
  // fb.group() : construye un FormGroup con los controles indicados.
  // Cada control es [valorInicial, listaDeValidadores].
  form = this.fb.group({
    titulo: ['', [Validators.required, Validators.minLength(3)]],
    isbn:   ['', Validators.required],
    autor:  ['', Validators.required],
    anio:   [2026, [Validators.min(1500), Validators.max(2100)]]
  });
  // form.valid : booleano que resume la validez de todos los controles.
  guardar() { if (this.form.valid) console.log(this.form.value); }
}
```

Signals: la primitiva signal()

Termino: Signal (reactividad de grano fino)

Contenedor reactivo de un valor tipado. Angular sabe que componente y que expresion de plantilla lee cada signal; cuando la signal cambia, actualiza solo las expresiones que la leyeron. Introducidas como estables en Angular 17 (2023).

contador con signal

```
import { signal } from '@angular/core';  
// signal<number>(0) : crea una signal de tipo number con valor inicial 0.  
const contador = signal<number>(0);  
  
// Lectura : se llama como funcion. Se devuelve el valor actual.  
console.log(contador());    // -> 0  
  
// Escritura absoluta : reemplaza el valor.  
contador.set(5);  
console.log(contador());    // -> 5  
  
// Escritura por transformacion : recibe el valor previo y devuelve el nuevo.  
contador.update(valor => valor + 1);  
console.log(contador());    // -> 6
```

computed(): estado derivado

Termino: `computed()`

Signal cuyo valor NO se asigna: se calcula a partir de otras signals. Angular vuelve a calcularla solo cuando alguna de las signals leidas cambia. Se memoiza: si nada cambia, no recalcula.

filtro derivado

```
import { signal, computed } from '@angular/core';
// Fuentes de datos: signals independientes.
const libros = signal<Libro[]>([]);
const filtro = signal<string>('');

// computed() : recibe una funcion que lee otras signals y devuelve un valor.
// Angular observa que signals se leyeron y crea la dependencia.
const librosFiltrados = computed(() => {
  const q = filtro().toLowerCase();
  return libros().filter(l => l.titulo.toLowerCase().includes(q));
});

// Uso : cada vez que cambia libros o filtro, librosFiltrados
// se recalcula automaticamente la proxima vez que se lea.
console.log(librosFiltrados().length); // recalcula si es necesario
console.log(librosFiltrados().length); // usa el valor memoizado
```

effect(): efectos colaterales

Termino: **effect()**

Registra una funcion que se ejecuta al montar el componente y cada vez que cambia alguna signal que lea dentro. Se usa para efectos colaterales: registrar en consola, guardar en localStorage, llamar a una API. NO se usa para calcular valores (para eso, computed).

efecto reactivo

```
import { signal, effect } from '@angular/core';

export class ContadorComponent {
  contador = signal(0);
  constructor() {
    // effect() : queda registrado en el inyector actual.
    // La funcion se dispara: (1) inmediatamente, (2) cada vez
    // que contador() cambie de valor.
    effect(() => {
      // Efecto colateral: guardar en el almacenamiento del navegador.
      localStorage.setItem('contador', String(this.contador()));
      console.log('contador cambio a', this.contador());
    });
  }
  incrementar() { this.contador.update(v => v + 1); }
}
```

Signals frente a Observables

Aspecto	Signal	Observable (RxJS)
Naturaleza	Contenedor sincrónico de un valor único	Flujo asincrónico de valores en el tiempo
Se lee con	Llamada como función: <code>mi.signal()</code>	<code>subscribe()</code> u operador <code>async pipe</code>
Se crea con	<code>signal(v)</code> , <code>computed(fn)</code> , <code>toSignal(obs)</code>	<code>new Observable()</code> , <code>of(v)</code> , <code>from(promise)</code>
Composición	<code>computed()</code> , <code>effect()</code>	<code>map</code> , <code>filter</code> , <code>mergeMap</code> , <code>switchMap</code> , <code>combineLatest</code>
Ideal para	Estado local del componente/formulario	Streams asincrónicos (HTTP, eventos, WebSocket)
En Angular 22	API estable y recomendada	Sigue estable; conviven ambos

Regla práctica: signal para estado; Observable para asincronía. HttpClient sigue devolviendo Observables; se convierten a signal con toSignal(observable) cuando conviene leerlos en la plantilla como signal.

Rutas anidadas (nested routes)

Termino: Ruta anidada

Ruta cuyo componente contiene otro <router-outlet> para renderizar sub-rutas. Permite construir vistas maestro-detalle o layouts con menu lateral persistente y contenido central que cambia.

app.routes.ts

```
// bibliotec-front/src/app/app.routes.ts
import { Routes } from '@angular/router';

// Cada objeto de la lista es una ruta.
// children : arreglo de rutas cuyo componente se renderiza
// dentro del <router-outlet> del componente padre.
export const routes: Routes = [
  { path: '', redirectTo: 'libros', pathMatch: 'full' },

  // /libros abre LibroLayoutComponent con menu lateral fijo
  { path: 'libros', component: LibroLayoutComponent, children: [
    { path: '', component: LibroListComponent },
    { path: ':id', component: LibroDetailComponent }, // /libros/42
    { path: 'nuevo', component: LibroFormComponent } // /libros/nuevo
  ]},
  { path: '**', component: NotFoundComponent } // ruta comodin (404)
];
```


Parametros de ruta y query params

leer paraetros

```
import { Component, inject, input, signal } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

@Component({ /* ... */ })

export class LibroDetailComponent {
  private servicio = inject(LibroService);
  private ruta      = inject(ActivatedRoute);

  // input() de router : Angular 16+ inyecta parametros como signal.
  // Requiere withComponentInputBinding() en provideRouter (angular.dev).
  id = input.required<number>();           // /libros/42 -> id() = 42
  edicion = input<string>('lectura');      // ?edicion=nueva

  // Alternativa clasica leyendo ActivatedRoute:
  libro = signal<Libro | null>(null);
  ngOnInit() {
    const rawId = this.ruta.snapshot.paramMap.get('id');
    if (rawId)
      this.servicio.buscarPorId(+rawId).subscribe(l => this.libro.set(l));
  }
}
```

Guards funcionales: canActivate

Termino: Guard (CanActivateFn)

Funcion que Angular ejecuta antes de activar una ruta y que devuelve true, false, o un UrlTree para redirigir. Se usa para restringir vistas a usuarios autenticados o a ciertos roles. Reemplaza a los guards basados en clase, ya obsoletos.

auth.guard.ts

```
// bibliotec-front/src/app/security/auth.guard.ts
// CanActivateFn : tipo de la funcion que Angular espera como guard.
// inject : lo usamos dentro para obtener el AuthService y el Router.
// Router.parseUrl : construye un UrlTree para redirigir con seguridad.
import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from '../auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  const router = inject(Router);
  // isAuthenticated() : signal booleana que expone el servicio.
  if (auth.isAuthenticated()) return true;
  // Si NO esta autenticado, se redirige a /login preservando la
  // URL destino para volver a ella tras iniciar sesion.
  return router.parseUrl(`/login?volver=${state.url}`);
};
```

canMatch y aplicacion a rutas

aplicar guards

```
// canMatch : variante que decide si la ruta siquiera se considera.  
// Si devuelve false, Angular sigue evaluando otras rutas (util con lazy).  
// canActivate : decide si el usuario puede acceder a una ruta ya elegida.  
import { CanMatchFn } from '@angular/router';  
  
export const adminMatch: CanMatchFn = () => {  
  const auth = inject(AuthService);  
  // hasRole('ADMIN') : consulta si el token JWT lleva ese rol.  
  return auth.hasRole('ADMIN');  
};  
  
// Aplicacion : el guard se anade al objeto de ruta.  
export const routes: Routes = [  
  { path: 'libros',      component: LibroListComponent,  
    canActivate: [authGuard] },  
  { path: 'usuarios',   component: UsuariosComponent,  
    canMatch: [adminMatch] },  
  { path: 'login',      component: LoginComponent },  
  { path: '**',         redirectTo: 'libros' }  
];
```

Lazy loading con loadComponent

Termino: Lazy loading (carga diferida)

Tecnica en la que un componente o modulo NO se incluye en el bundle inicial que descarga el navegador, sino en un bundle separado que se descarga solo cuando el usuario navega a esa ruta. Reduce el tiempo de carga inicial.

cargar bajo demanda

```
// Sin lazy loading (todo el bundle desde el arranque):
import { LibroListComponent } from './libros/libro-list.component';
{ path: 'libros', component: LibroListComponent }
// Con lazy loading (componente en bundle separado):
// loadComponent : funcion que recibe una funcion asincrona que
// hace import() dinamico y devuelve el componente.
// El bundler (esbuild) crea un chunk aparte para ese modulo.
{ path: 'libros',
  loadComponent: () =>
    import('./libros/libro-list.component')
      .then(m => m.LibroListComponent) },
// Tambien existe loadChildren para cargar un grupo de rutas hijas:
{ path: 'admin',
  loadChildren: () =>
    import('./admin/admin.routes').then(m => m.ADMIN_ROUTES) }
```

OnPush: el nuevo default en Angular 22

Termino: Change detection (deteccion de cambios)

Proceso por el que Angular decide que expresiones de la plantilla se re-evaluan cuando algo cambia. Con la estrategia OnPush, el componente solo se re-renderiza cuando sus inputs cambian por referencia o cuando una signal que lea cambia.

OnPush por defecto

```
// En Angular 22, si un componente NO declara changeDetection,  
// el default es ChangeDetectionStrategy.OnPush (antes: Default).  
import { Component, ChangeDetectionStrategy, signal } from '@angular/core';  
  
@Component({  
  selector: 'app-libro-card',  
  standalone: true,  
  template: `<h3>{{ libro().titulo }}</h3>`,  
  // Explicito (opcional en v22). Si se omite, Angular usa OnPush.  
  changeDetection: ChangeDetectionStrategy.OnPush  
})  
  
export class LibroCardComponent {  
  libro = signal<Libro>({ /* ... */ });  
}
```

@defer: vistas diferidas por gatillo

Termino: Vista diferida (@defer block)

Bloque de plantilla cuya carga y renderizado se posterga hasta que ocurre un gatillo (on hover, on interaction, on viewport, on idle, on timer). Aparece como estable desde Angular 17. Reduce el trabajo del hilo principal en la primera carga.

defer con on viewport

```
<!-- Bloque @defer : el componente pesado se carga solo cuando -->
<!-- el elemento marcado con #trigger entra en el viewport.      -->
<div>
  <h2>Detalle del libro</h2>
  <p>{{ libro().titulo }}</p>
  <hr #trigger />

  <!-- on viewport(trigger) : dispara cuando el ref entra a viewport -->
  @defer (on viewport(trigger)) {
    <app-comentarios [libroId]="libro().id" />
  }
  @placeholder { <p>Los comentarios se cargaran al bajar...</p> }
  @loading      { <p>Cargando comentarios...</p> }
  @error        { <p>No se pudieron cargar los comentarios.</p> }
}
</div>
```

Testing: Vitest como default

Termino: Vitest

Corredor de pruebas basado en Vite, adoptado como default por Angular CLI desde la version 21. Compatible con la sintaxis de Jasmine (describe, it, expect). Ofrece arranque mas rapido que Karma y no requiere navegador para las pruebas de logica.

Al crear un proyecto Angular 22 con `ng new`, la CLI configura Vitest automaticamente. Al ejecutar '`ng test`' la salida indica el runner y las pruebas.

terminal

```
# Ejecutar la suite de pruebas del proyecto
ng test

# Salida esperada (abreviada):
# RUN v3.x.x /home/dev/biblioteq/frontend
# DEV Server running at http://localhost:5173/
# Test Files 2 passed (2)
# Tests 7 passed (7)
# Duration 1.34s
```

Prueba de un servicio con HttpClient

libro.service.spec.ts

```
// bibliotec-front/src/app/libros/libro.service.spec.ts
// describe : agrupa un conjunto de pruebas relacionadas.
// it : declara una prueba individual con nombre humano.
// expect : declara una asercion que debe cumplirse.
import { describe, it, expect, beforeEach } from 'vitest';
import { TestBed } from '@angular/core/testing';
// provideHttpClient : registra HttpClient en el TestBed.
// provideHttpClientTesting : sustituye la implementacion
// por una que intercepta y controla las llamadas HTTP.
import { provideHttpClient } from '@angular/common/http';
import { HttpTestingController, provideHttpClientTesting } from '@angular/common/http/testing';
import { LibroService } from '../libro.service';
describe('LibroService', () => {
  let servicio: LibroService;
  let http: HttpTestingController;
  beforeEach(() => {
    // se ejecuta antes de cada it()
    TestBed.configureTestingModule({ providers: [ provideHttpClient(), provideHttpClientTesting() ] });
    servicio = TestBed.inject(LibroService);
    http = TestBed.inject(HttpTestingController);
  });
  it('lista libros del backend', () => {
    // subscribe : dispara la peticion HTTP.
    servicio.listar().subscribe(libros =>
      expect(libros.length).toBe(2));
    // expectOne : verifica que se hizo una unica peticion a la URL.
    const req = http.expectOne('http://localhost:8080/api/libros');
    req.flush([ { id:1, titulo:'A' }, { id:2, titulo:'B' } ]);
  });
});
```


Prueba de un componente con TestBed

Termino: TestBed

Utilidad oficial de Angular para montar componentes, directivas o servicios en un entorno aislado de pruebas. Reemplaza providers, permite compilar plantillas y crear fixtures (envoltorios del componente para inspeccionar y actuar).

prueba de componente

```
import { describe, it, expect } from 'vitest';
import { TestBed } from '@angular/core/testing';
import { LibroCardComponent } from './libro-card.component';

describe('LibroCardComponent', () => {
  it('renderiza el titulo del libro', () => {
    // createComponent : instancia el componente y su plantilla.
    const fixture = TestBed.createComponent(LibroCardComponent);
    fixture.componentRef.setInput('libro',
      { id:1, titulo:'Ficciones' });
    // detectChanges : corre la deteccion de cambios manualmente.
    fixture.detectChanges();
    // nativeElement : nodo del DOM raiz del componente.
    const html = fixture.nativeElement.textContent as string;
    expect(html).toContain('Ficciones');
  });
});
```

Dockerfile multi-stage del frontend

Termino: Multi-stage build

Estrategia de Docker en la que un Dockerfile declara varias FASES (stage). La primera compila la aplicación en un contenedor con todas las herramientas. La segunda copia SOLO el resultado a una imagen mínima que se despliega. El artefacto final pesa mucho menos que el de compilación.

Dockerfile

```
# bibliotecq/frontend/Dockerfile
# --- Fase 1: compilacion con Node.js 24 ---
FROM node:24-alpine AS build
WORKDIR /app                                # directorio de trabajo dentro del contenedor
COPY package*.json ./                      # copiar solo el manifiesto
RUN npm ci                                 # instalacion reproducible desde el lock
COPY . .                                    # copiar el resto del codigo
RUN npx ng build --configuration=production # genera dist/frontend/browser

# --- Fase 2: servir con nginx (imagen final delgada) ---
FROM nginx:1.27-alpine AS runtime
COPY --from=build /app/dist/frontend/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

nginx.conf con proxy_pass a la API

Termino: proxy_pass

Directiva de nginx que reenvia una petición HTTP a otro servidor. Sirve para que el frontend (puerto 80) y la API (puerto 8080) parezcan servidos desde el mismo origen y eviten CORS en producción.

nginx.conf

```
# bibliotecq/frontend/nginx.conf
server {
    listen 80;                        # escucha HTTP
    server_name bibliotecq.uteq.edu.ec; # nombre publico
    # Angular SPA : sirve los estaticos del build.
    root /usr/share/nginx/html;
    index index.html;
    # try_files : para rutas de Angular Router (fallback a index.html).
    location / {
        try_files $uri $uri/ /index.html;
    }
    # Reverse proxy : /api/* se reenvia al backend Spring Boot.
    location /api/ {
        proxy_pass http://backend:8080/api/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

docker-compose.yml del monorepo

docker-compose.yml

```
# bibliotecq/docker-compose.yml
# services : declara los contenedores a levantar juntos.
# Cada servicio expone puertos y depende de otros por nombre.
services:

  db:                                # servicio de base de datos

    image: postgres:16-alpine

    environment:

      POSTGRES_DB: biblioteca

      POSTGRES_USER: biblio

      POSTGRES_PASSWORD: cambiar123

    ports:

      - '5432:5432'

  backend:                           # servicio Spring Boot

    build: ./backend-java
```

Build de produccion optimizado

terminal

```
# En la carpeta frontend/  
ng build --configuration=production  
  
# Resultado en dist/frontend/browser (~200-500 kB)  
# El bundle esta minificado, con hashes en los nombres para invalidar cache.
```

Optimizaciones activadas

Compilacion AOT (Ahead-of-Time) que precompila las plantillas. Tree-shaking que elimina codigo no importado. Minificacion de JS y CSS. Hashing de nombres para invalidacion de cache HTTP. Reduccion via Brotli o gzip en el servidor.

Budgets configurables

En angular.json se declaran budgets: limites de tamano por bundle inicial (default 500 kB) y por bundle de componente (4 kB). Superarlos genera warning; superar los limites 'error' hace fallar el build. Fuerza a mantener la app ligera.

Checklist previo a produccion

- `environment.prod.ts` apunta a la URL publica de la API y NO al localhost.
- `fileReplacements` declarado en `angular.json` para sustituir `environment` por `environment.prod` al construir con `--configuration=production`.
- Interceptor JWT registrado y `AuthService` que refresca el token con antelacion al vencimiento.
- CORS resuelto en el backend con la URL publica del frontend (o servidos desde el mismo origen mediante `proxy_pass`).
- Pruebas unitarias verdes con `'ng test --watch=false'` antes de fusionar a main.
- Dockerfile multi-stage compilado y verificado localmente con `docker build` y `docker run`.
- `docker-compose.yml` probado con `docker compose up` y verificado con `curl http://localhost/api/libros`.
- Etiqueta Git firmada segun SemVer (por ejemplo, `v1.0.0`) creada en el commit de release.

Directiva de atributo personalizada

Termino: Directiva

Clase decorada con `@Directive` que altera el aspecto o comportamiento de un elemento HTML. Se aplica como atributo del elemento y no crea DOM propio, a diferencia del componente.

directiva

```
// bibliotecq-front/src/app/shared/marcar-invalido.directive.ts
import { Directive, ElementRef, inject, effect, input } from '@angular/core';
@Directive({
  selector: '[appMarcarInvalido]',          // usar como <input appMarcarInvalido>
  standalone: true
})
export class MarcarInvalidoDirective {
  // input() : declara un input reactivo (signal).
  invalido = input.required<boolean>();

  // ElementRef : referencia al nodo HTML donde se aplico la directiva.
  private el = inject(ElementRef<HTMLElement>);
  constructor() {
    // effect() : cada vez que invalido() cambie, actualizamos el estilo.
    effect(() => {
      const cls = this.el.nativeElement.classList;
      cls.toggle('borde-rojo', this.invalido());
    });
  }
}
```

Pipe personalizado

Termino: Pipe

Funcion que transforma un valor para mostrarlo en la plantilla. Se aplica con el operador |. Ejemplo del framework: {{ fecha | date:'shortDate' }}. Un pipe personalizado permite encapsular una transformacion propia del dominio.

pipe personalizado

```
// bibliotec-front/src/app/shared/dias-vencido.pipe.ts
// Pipe : decorador que marca la clase como pipe.
// PipeTransform : interfaz que obliga a implementar transform().
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'diasVencido', standalone: true, pure: true })

export class DiasVencidoPipe implements PipeTransform {
  // transform(fechaVencimiento) devuelve el numero de dias vencidos.
  // pure:true : Angular solo recomputa cuando cambia el argumento.
  transform(vencimiento: Date | string): number {
    const v = typeof vencimiento === 'string' ? new Date(vencimiento) : vencimiento;
    const hoy = new Date();
    const ms = hoy.getTime() - v.getTime();
    return Math.max(0, Math.ceil(ms / 86_400_000)); // 24h en ms
  }
}

// Uso en plantilla:
// {{ prestamo.fechaVencimiento | diasVencido }} dias vencido
```


Buenas practicas en el proyecto BiblioUTEQ

- Feature folder : agrupar por característica de negocio (libros/, prestamos/, usuarios/) y no por tipo (services/, components/).
- Componentes tontos y contenedores : el componente contenedor conoce el servicio y los datos; los tontos reciben inputs y emiten eventos.
- Un servicio por agregado : LibroService, PrestamoService, UsuarioService, con inject() de HttpClient.
- Model tipado en un .model.ts junto al servicio; interface Libro reflejando el JSON del backend.
- Rutas con lazy loading para las secciones grandes o rara vez visitadas (por ejemplo, administracion).
- Guards funcionales para proteger vistas por rol (authGuard, adminMatch).
- Interceptor de JWT y de manejo de errores en @angular/common/http registrados en app.config.ts.
- Pruebas unitarias con Vitest y cobertura minima del 70 por ciento en servicios.

Recursos oficiales y comunidad

- Documentacion oficial de Angular 22 en <https://angular.dev> con tutorial interactivo y guias tema por tema.
- Blog oficial en <https://blog.angular.dev> con anuncios de version, roadmap y ejemplos verificados.
- Repositorio publico en <https://github.com/angular/angular> con issues, releases y codigo fuente.
- Angular Live en YouTube (canal oficial) con sesiones tecnicas y AMAs con miembros del equipo.
- Meetup local Angular Ecuador para networking y ejercicios en grupo.

Sobre tutoriales de terceros

Antes de seguir un tutorial no oficial, verificar la fecha: cualquier material anterior a mayo de 2024 (Angular 18) usa patrones desactualizados (NgModule, ChangeDetection default eager, HttpClientModule).

Mapa completo de lo entregado

Entre el deck 1 y este deck 2, el aprendiz sale de la Unidad IV con el siguiente conjunto de conocimientos y artefactos en su repositorio.

Setup y estructura

- Node 24 LTS + Angular CLI 22
- IntelliJ IDEA Ultimate configurado
- Proyecto frontend/ generado
- Rutas con Router + guards

Consumo de la API REST

- HttpClient con provideHttpClient
- GET/POST/PUT/DELETE completos
- Interceptor JWT operando
- environments por entorno

Formularios y estado

- Signal Forms para nuevos formularios
- Schema declarativo con validadores
- computed() y effect() en la vista
- Signals de campo con errores

Produccion

- Vitest para pruebas unitarias
- Multi-stage Dockerfile
- nginx con proxy_pass
- docker-compose completo

Roadmap del resto de la Unidad IV

- Semana 15 : documentacion viva de la API con OpenAPI 3.1 y Swagger UI --- integra el JSON de la API en el propio frontend.
- Semana 16 : consumo de servicios externos (mapas, correos transaccionales, notificaciones push).
- Semana 17 : monitoreo del sistema en produccion (logs estructurados, healthchecks, metricas).
- Semana 18 : cierre y entrega final --- integracion completa, defensa oral del PFC y entrega de la carpeta docs/entregas/entrega-final.
- Entregable transversal : la Guia PFC Entrega 2B (recasteo de la Entrega Final) al cierre de la Unidad IV.

Referencias verificadas

- [1] Angular Team. Announcing Angular v22. blog.angular.dev, 3 jun. 2026.
- [2] Google LLC. Angular Documentation --- Forms (Signal Forms). angular.dev/guide/forms (2026).
- [3] Google LLC. Angular Documentation --- Signals. angular.dev/guide/signals (2026).
- [4] Google LLC. Angular Documentation --- Router Guards. angular.dev/guide/routing (2026).
- [5] Google LLC. Angular Documentation --- Deferrable Views. angular.dev/guide/templates/defer (2026).
- [6] Google LLC. Angular Documentation --- Migrating to Vitest. angular.dev/guide/testing/migrating-to-vitest (2026).
- [7] Vitest Team. Vitest --- Blazing Fast Unit Test Framework. vitest.dev (2026).
- [8] OpenJS Foundation. Node.js 24 LTS Documentation. nodejs.org/docs/latest-v24.x (jul. 2026).
- [9] Docker Inc. Docker Documentation. docs.docker.com (2026).
- [10] Nginx Inc. Nginx Documentation. nginx.org/en/docs/ (2026).
- [11] Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures. Thesis, UC Irvine, 2000.
- [12] Fielding, R.; Nottingham, M.; Reschke, J. HTTP Semantics. IETF RFC 9110. Jun. 2022. doi:10.17487/RFC9110.
- [13] Jones, M.; Bradley, J.; Sakimura, N. JSON Web Token (JWT). IETF RFC 7519. May 2015. doi:10.17487/RFC7519.
- [14] Freeman, A. Pro Angular: Build Powerful and Dynamic Web Apps. 5.a ed. Apress, 2023. isbn 978-1-4842-9426-1.

Angular 22 --- listo para produccion

Que se logro con esta continuacion

1. Formularios con Signal Forms, validacion declarativa y submit tipado.
2. Estado del componente con signals, computed y effect.
3. Rutas con guards funcionales y lazy loading por seccion.
4. Pruebas unitarias con Vitest para servicios y componentes.
5. Despliegue con Dockerfile multi-stage y nginx enfrente de la API.

Que sigue en la Unidad IV

Documentacion OpenAPI/Swagger, consumo de servicios externos, monitoreo en produccion y defensa final del PFC.

Preguntas

Aplicaciones Web --- UTEQ --- PPA 2026-2027

Dr. Gleiston Ciceron Guerrero Ulloa, Ph.D. --- gguerrero@uteq.edu.ec